



M6800AS(D)
Jan., 1977

M6800

BASIC INTERPRETER
REFERENCE MANUAL

The information in this manual has been carefully reviewed and is believed to be entirely reliable. However, no responsibility is assumed for inaccuracies. Furthermore, such information does not convey to the purchaser of the product described any license under any patent and trademark registration rights and copyrights of Motorola, Inc., or others.

The material in this manual is subject to change, and Motorola, Inc., reserves the right to change specifications without notice.

EXORciser and EXbug are trademarks of Motorola, Inc.

First Edition
Copyright 1977 by Motorola, Inc.

Printed in Switzerland 108-077/1

TABLE OF CONTENTS

CHAPTER 1:	GENERAL DESCRIPTION	1-1
1.1	INTRODUCTION	1-1
1.2	FEATURES	1-1
1.3	PROGRAM STRUCTURE	1-2
1.3.1	Statements	1-2
1.3.2	Commands	1-3
1.3.3	Intrinsic Functions	1-3
1.4	DATA FORMATS	1-3
1.4.1	Numbers	1-3
1.4.2	Simple Variables	1-3
1.4.3	String Variables	1-3
1.4.4	Constants	1-4
1.5	PROGRAM MODES	1-4
1.5.1	Command Mode	1-4
1.5.2	Execution Mode	1-4
CHAPTER 2:	PROGRAM PREPARATION	2-1
2.1	LOADING THE SOURCE PROGRAM	2-1
2.2	CHANGING THE SOURCE PROGRAM	2-2
CHAPTER 3:	STATEMENTS	3-1
3.1	INTRODUCTION	3-1
3.2	INPUT/OUTPUT STATEMENTS	3-1
3.2.1	INPUT Statement	3-1
3.2.2	DATA and READ Statements	3-2
3.2.3	RESTORE Statement	3-4
3.2.4	PRINT Statement (Direct)	3-4
3.3	DECLARATION STATEMENTS	3-5
3.3.1	DIMension Statement	3-5
3.4	CONTROL STATEMENTS	3-6
3.4.1	FOR and NEXT Statements	3-6
3.4.2	STOP Statement	3-7
3.4.3	END Statement	3-7
3.4.4	GOTO Statement (Direct)	3-7
3.4.5	GOSUB Statement	3-7
3.4.6	RETURN Statement	3-8
3.4.7	ON Statement	3-8
3.4.8	IF Statement (BASIC Statement - Direct)	3-9
3.5	ASSIGNMENT STATEMENTS	3-9
3.5.1	LET Statement (Direct)	3-10
3.5.2	Mathematical Operator Statements	3-10
3.5.3	Operation Priority Statements	3-11
3.5.4	String Concatenation Statements	3-11
3.6	REMARKS STATEMENTS	3-12
3.6.1	REM Statement	3-12

CHAPTER 4:	COMMANDS	4-1
4.1	INTRODUCTION	4-1
4.2	SYSTEM COMMANDS	4-1
4.2.1	LOAD Command	4-1
4.2.2	APPEND Command	4-1
4.2.3	SAVE Command	4-1
4.2.4	EXIT Command	4-1
4.2.5	POKE Command	4-2
4.2.6	LIST Command	4-2
4.2.7	TRACE ON Command	4-3
4.2.8	TRACE OFF Command	4-3
4.3	PROGRAM EXECUTION COMMANDS	4-3
4.3.1	RUN Command	4-3
4.3.2	CONT Command	4-4
4.3.3	NEW Command	4-4
4.4	EDITING COMMANDS	4-4
4.4.1	Control C Command	4-4
4.4.2	Control X Command	4-4
4.4.3	Rub Out Command	4-4
4.4.4	PATCH Command	4-5
4.5	SIZING COMMANDS	4-5
4.5.1	LINE Command	4-5
4.5.2	DIGITS Command	4-6
CHAPTER 5:	INTRINSIC FUNCTIONS	5-1
5.1	INTRODUCTION	5-1
5.2	CONTROL FUNCTIONS	5-1
5.2.1	TAB Function	5-1
5.2.2	POS Function	5-1
5.3	DATA FUNCTIONS	5-2
5.3.1	RND Function	5-2
5.3.2	INT Function	5-2
5.3.3	ABS Function	5-2
5.3.4	SGN Function	5-3
5.3.5	LEN Function	5-3
5.3.6	ASC Function	5-3
5.3.7	CHR\$ Function	5-3
5.3.8	VAL Function	5-4
5.3.9	STR\$ Function	5-4
5.3.10	LEFT\$ Function	5-4
5.3.11	RIGHT\$ Function	5-4
5.3.12	MID\$ Function	5-5
5.3.13	PEEK Function	5-5
5.4	MATHEMATICAL FUNCTIONS	5-5
5.5	USER DEFINED FUNCTION	5-6
5.5.1	DEF Function	5-6

APPENDIX A:	GENERAL CONVERSION CHARTS	A-1
APPENDIX B:	ASCII CONVERSION CHART	B-1
APPENDIX C:	EXbug COMMANDS	C-1
APPENDIX D:	SAMPLE PROGRAM	D-1
APPENDIX E:	ERROR MESSAGES	E-1
APPENDIX F:	ENHANCING PROGRAM OPERATION	F-1
APPENDIX G:	MINIMIZING DEDICATED MEMORY	G-1
APPENDIX H:	BASIC INTERPRETER SUMMARY	H-1

CHAPTER 1

GENERAL DESCRIPTION

1.1 INTRODUCTION

The EXORciser-resident BASIC Interpreter provides a new tool for problem solving to the M6800 Microcomputer family of parts. BASIC is a high-level programming language widely used for general purpose and certain business-related applications. The system consists of the BASIC software and the following hardware:

- EXORciser
- 8K bytes of memory
- TTY or RS-232C compatible terminal with cassette drive unit
- EXOrtape or EXORDisk drive unit (if cassette option is not chosen)

The minimum configuration may be expanded to include up to 65,536 bytes of memory (less 4096 bytes which are reserved for the EXbug firmware program) and up to four disk drives for a total of over 1 million bytes of disk storage capability. In addition, a variety of hard copy line printers and CRT terminal may be used. Throughout this manual, it has been assumed that a terminal having both a keyboard and printer is being used. If you are using a CRT terminal instead, the words "terminal printer" may be replaced by the words "CRT display".

1.2 FEATURES

The features of the BASIC Interpreter program are:

- All mathematical operations are performed in BCD (Binary Coded Decimal) arithmetic for maximum accuracy.
- User programs may be saved and loaded from cassette, paper tape, or floppy disk.
- Most arithmetic functions and transcendentals are implemented as directly executable subprograms.
- String variables and arrays are permitted.
- Most program statements may be executed in the direct mode (no statement numbers required for immediate calculations).
- The BASIC Interpreter program requires about 7600 bytes of memory, thus limiting the size of user programs that can be executed with a minimum configuration (8K). More memory can, however, be added if the user program requires.

1.3 PROGRAM STRUCTURE

A user program written in BASIC consists of various statements, commands, and intrinsic functions. The following paragraphs provide a brief description of each of these parts, while chapters referenced within each paragraph provide an in-depth description.

1.3.1 Statements

Every statement begins with a statement number, followed by the statement directive (or name) and statement argument, and terminated by a Carriage Return (CR). Five types of statements are used in BASIC: Input/Output, Declaration, Control, Assignment, and Remarks. These statements are described in Chapter 3. The following rules apply to all BASIC statements.

- Every statement must have a statement number ranging between 1 and 9999. DO NOT USE LINE NUMBER 0.
- Statement numbers are used to sequentially order the program statements.
- In any program, a statement number may only be used once.
- A previously entered line may be changed by entering the same line number along with the desired new statement. Entering a line number immediately followed by a Carriage Return deletes that line.
- Statements do not need to be entered in numeric sequence, since an ascending order will automatically be established by the BASIC Interpreter program.
- A statement may NOT contain more than 72 characters, including spaces.
- Spaces, unless used within a character string and enclosed within quotation marks, are NOT processed. Thus, the use of spaces is optional. It enhances readability, but takes longer to process. Numbers MUST NOT contain embedded spaces. All examples presented in this manual include spaces.

EXAMPLE: 110 LET A=B + (3.5*5E2)
 is exactly equal to:
 110LETA=B+(3.5*5E2)

1.3.2 Commands

Commands (and statements used as commands) are statements which, when issued at the terminal keyboard by the user, are immediately executed by the BASIC program. All commands are issued without the need for statement numbers. All of the commands used in the BASIC program are described in Chapter 4.

1.3.3 Intrinsic Functions

Intrinsic functions are program functions which are inherent to the BASIC Interpreter program itself and do not require a separate program for execution. Four types of intrinsic functions are available to the user: control, data, mathematical, and user defined. The intrinsic functions are described in Chapter 5.

1.4 DATA FORMATS

All data entries used with the BASIC program must be formatted in accordance with the following paragraphs.

1.4.1 Numbers

The range of numbers capable of being represented in BASIC is: 1.0E-99 to 9.99999999E+99. The "E" represents exponent; thus, E-99 equals 10^{-99} and E+99 equals 10^{+99} . All numbers entered which exceed these limitations are internally truncated (last digits dropped) to nine significant digits. Numbers may be entered and displayed in either of three formats: integer, decimal, or exponential.

EXAMPLE: 153 (integer)
 153.34 (decimal)
 153E-2 (exponential)

1.4.2 Simple Variables

Simple variables may be named any single alphabetic character or any single alphabetic character followed by a number (0 through 9).

EXAMPLE: A, B, C, etc.
 A3, B4, C5, etc.

1.4.3 String Variables

String variables are indicated by any single alphabetic character or subscripted alphabetic character followed by a dollar sign (\$) character. String variables may contain a MAXIMUM of 18 characters, including spaces. Even if only one character is contained within the string variable, 18 bytes will be reserved. Each character in a string is represented internally in ASCII.

EXAMPLE: A\$, B\$, C\$, etc.
 A\$(3), B\$(4), C\$(5), etc.

C\$ = " " Value

34.52
.554
136E-2

1.5 PROGRAM MODES

Two operating modes are used during entering and executing BASIC programs: the command mode and the execution mode.

1.5.1 Command Mode

The system operates in the command mode when it is ready for the user to enter the next program statement or command. The system indicates the command mode to the user by printing the # character as a prompt.

1.5.2 Execution Mode

The system operates in the execution mode during program execution.

CHAPTER 2

PROGRAM PREPARATION

2.1 LOADING THE SOURCE PROGRAM

The BASIC source program is available on any one of three types of storage media: cassette, paper tape, or diskette. Loading the source program from the storage media into memory involves an inter-active entry process - that is, prompts are provided by the loading routine on the terminal printer and responses to these prompts are entered by the operator via the terminal keyboard.

The procedure required to load the BASIC Interpreter program into memory from the storage media is dependent on the type of storage media used. If the BASIC Interpreter program is stored on cassette or paper tape, then program loading is controlled by the EXbug MAID routine. Therefore, after the user accesses the MAID routine, he must then enter memory location 100 to load the BASIC Interpreter program (100;G). However, if the BASIC Interpreter program is stored on diskette, all the user must do to load the program is enter the word EDIT, followed by the Old File name (if required) and the New File name. This procedure is more fully discussed in Example B. After the BASIC Interpreter program is loaded, the word READY is printed on the terminal printer followed by the # character on the next line (refer to the following examples). This indicates that the program is ready to accept inputs (either program statements or commands) from the terminal keyboard. After each successive program statement is entered, the system will prompt the user by printing a # character. This indicates that the statement has been entered. After each command, the word READY will again be printed followed by the # character. This indicates that the command has been executed and the system is ready to input the next program statement or command.

Example A shows the inter-active procedure used to load source programs from cassette or paper tape into memory. Example B shows the procedure to load source programs into memory from diskette. In both of these examples, the proper operator responses are shown underlined.

EXAMPLE A (cassette or paper tape only):

```
EXBUG X.X LOAD
36L/CONT S
BASIC8
EXBUG X.X MAID
*100;G
READY
#
```

NOTE: The # character indicates that the BASIC source program has been completely loaded and is ready to accept inputs from the terminal keyboard.

EXAMPLE B (diskette only):

```
!BASIC,OF,NF
M5800 RESIDENT BASIC 1.0
COPYRIGHT(C) - 1976
READY
```

NOTES: " OF = Name of Old File user wishes to load.

NF = New File name the user wishes to create.
If the user enters only the words EDIT,,NF,
a new file is created. However, if the user
enters the words EDIT,OF,NF at the beginning,
then the Old File is loaded and updated or
modified prior to being saved under the New
File name.

= indicates that the BASIC program and the
Old File have been completely loaded and
BASIC is ready to accept inputs.

2.2 CHANGING THE SOURCE PROGRAM

After the BASIC Interpreter program has been loaded into memory, the range of working memory is automatically determined by the program. The system is now ready to accept commands or statements from the terminal keyboard. For example, the user may wish to enter the following program:

```
READY
#10 REM DEMONSTRATION
#20 PRINT "ENTER A NUMBER"
#30 INPUT R
#40 LET A=(R*R)*3.1415926
#50 PRINT
#60 PRINT "THE AREA OF A CIRCLE WITH ";
#70 PRINT "RADIUS ";R;"IS ";A
```

If the user wishes to insert a statement between two others, he types a statement number that falls between the other two followed by the statement he wishes to insert. After the statement has been completely entered, the user enters a Carriage Return to complete the insert. The following example shows this type of inserted statement:

```
#45 REM THIS INSERTED BETWEEN 40 AND 50
```

If the user wants to replace a statement, the statement number of the statement to be replaced must be typed, followed by the new statement and a Carriage Return. For example:

```
#50 PRINT "A=";A
```

Each line entered must be terminated by a Carriage Return. The BASIC program automatically advances the printer to the start of the next line when a Carriage Return is recognized.

When the CTRL (control key) and the X key are simultaneously depressed, the program will delete the line; depressing the "RUBOUT" key causes the program to backspace one character. A further explanation is provided in chapter 4.

If the user is now ready to execute the program to this point, the RUN command should be entered on the terminal keyboard.

CHAPTER 3

STATEMENTS

3.1 INTRODUCTION

All BASIC program statements consist of a symbol or group of symbols representing a variable or group of variables combined by symbols representing operators in accordance with a set of definitions and rules established by the BASIC Interpreter program. Five categories of statements are used with the BASIC Interpreter: input/output, declaration, control, assignment, and remarks. These statements, when entered, are stored in memory and are not executed until an execute command is entered by the user (refer to Chapter 4 for the various commands). However, when the BASIC program is in the command mode (waiting for a user input), some statements may be entered without statement numbers. The BASIC program will execute these statements immediately. This is referred to as the direct mode of execution. Since statements entered in the direct mode are normally of a mathematical nature, by using the direct mode of execution the user obtains the immediate solution to problems without requiring iterative (repeated) program procedures. Thus, by using a directly executable statement, the BASIC Interpreter program may be used in a manner very similar to an electronic calculator. In the following paragraphs, all of the BASIC Interpreter program statements are listed and explained by category. Only those statements which are labeled "direct" may be used in the direct execution mode.

3.2 INPUT/OUTPUT STATEMENTS

Input/output statements are used to control the flow of data between the EXORciser and the input/output peripherals. The following paragraphs list and explain each of the input/output statements.

3.2.1 INPUT Statement

The INPUT statement permits the user to enter data from the terminal keyboard during program execution. Example A presents two typical INPUT statements.

EXAMPLE A:

INPUT X	means	input one numeric value (X)
---------	-------	-----------------------------

INPUT X\$	means	input one string variable (X\$)
-----------	-------	---------------------------------

13

When an INPUT statement occurs in the program, a question mark (?) character is printed on the terminal printer. In response to this prompt, the user types in the requested data, separated by commas, and followed by a Carriage Return. Constants and strings can be entered in response to an INPUT statement. If insufficient data is entered by the user, the system will once again prompt with a ? character. If no data is entered or if a non-numeric value is entered into a variable, the system will prompt with the word RE-ENTER. Example B presents the method of entering both simple variables and string variables using the INPUT statement. Note that each of the variables are separated by a comma (,) character.

EXAMPLE B:

INPUT X,Y,Z,A\$

Multiple inputs may be entered. If the expected number of values are not entered, the system will prompt with another ? character.

INPUT "ENTER VALUE",X

Prints the message enclosed within quotation marks as a prompt to the user. A ? character is then printed directly after the message. The system then awaits the entered value. This value is stored in argument X.

3.2.2 DATA and READ Statements

The DATA and READ statements are used in conjunction with each other to provide a method of assigning values to variables (using the LET statement is still another way).

The DATA statement sequentially loads each value specified in the argument field (the field of data directly following the DATA directive) into the next available memory location within the data buffer. Each value within the argument field is separated from the next (delimited) by a comma (,) character. If string variables are used within the argument field, they do not need to be enclosed within quotation marks, since the comma acts as the delimiter. However, if a comma is used within the string variable, then the string variable must be enclosed within quotation marks. All DATA statements, regardless of where they occur in the program, cause the DATA argument to be combined into one list (instead of separate lists as would be the case if LET statements were used).

The READ statement sequentially accesses the values previously loaded into the data buffer by the DATA statement and assigns these values to the variables specified in the argument field of the READ statement. Each variable specified within the READ statement argument field is

separated from the next (delimited) by a comma (,) character. When entering READ statements, both simple variables and string variables may be used intermixed. However, if this is done, the simple variables and string variables must be entered in the appropriate sequence with respect to the values entered in the DATA statements. A typical example of a DATA and READ statement is presented in Example A. This example includes both simple and string variables.

EXAMPLE A:

```
#110 DATA 10,20,30,45.5,"TEST, ONE",1.2345,6.7E9,END
#120 READ A,B,C,D,E,F,G4,F4
```

Every time a READ statement is encountered in the program, the value stored in the first available memory location of the data buffer by the first argument of the first DATA statement is assigned the first variable specified by the first argument of the READ statement. The next value stored in the second available memory location of the data buffer by the second argument of the first DATA statement is assigned the second variable specified by the second argument of the READ statement. This continues until the last value stored in the data buffer by the last argument of the first DATA statement is assigned a variable specified by the READ statement. The first argument of the second DATA statement is then assigned to the next variable specified by the READ statement. This process continues until the last variable specified by the READ statement is assigned a value in the DATA statement. If the last argument(s) of the READ statement is not assigned (more variables in the READ statement than values in the DATA statement), an error message would be generated (error message 19). It is, however, permissible to have more values in the DATA statement than variables in the READ statement. Example B shows a typical DATA statement and READ statement and the equivalent method of assigning values to variables using the LET statement.

EXAMPLE B:

```
#110 DATA 1,2,3
#120 DATA 4,5,6
#130 DATA 7,8,9
#140 READ A,B,C,D,E,F,G,H,J
```

Is the equivalent of:

```
#110 LET A=1
#120 LET B=2
#130 LET C=3
#140 LET D=4
#150 LET E=5
#160 LET F=6
#170 LET G=7
#180 LET H=8
#190 LET J=9
```

3.2.3 RESTORE Statement

The RESTORE statement resets the data buffer pointer, which is advanced by the execution of READ statements, to the first position in the data buffer. This permits the first argument of the first DATA statement to be reassigned to the first variable of the next READ statement. The example below shows how the RESTORE statement can be used and also presents the equivalent of the example if LET statements were used.

EXAMPLE:

```
#110 DATA 1,2,3
#120 DATA 4,5,6
#130 DATA 7,8,9
#140 READ A,B,C
#150 RESTORE
#160 READ D,E,F
#170 RESTORE
#180 READ G,H,I
```

Is the equivalent of:

```
#110 LET A=1
#120 LET B=2
#130 LET C=3
#140 LET D=1
#150 LET E=2
#160 LET F=3
#170 LET G=1
#180 LET H=2
#190 LET I=3
```

3.2.4 PRINT Statement (Direct)

The PRINT statement enables the user to tell the BASIC program to print the value of expressions, literal values, simple values, or test strings on the user's terminal printer or line printer. The various directives may be separated in the PRINT statement by using either comma (,) characters or semicolon (;) characters. If commas are used to separate the various directives, each directive printed will be separated into 16 character space zones. If more than four directives are separated by commas (requiring more than 72 characters), the remaining directives (in groups of 4) will be printed on the following line(s). If semicolons are used to separate the various directives, only one space character will be inserted between each directive printed. No space is produced if a variable string is entered and followed by a semicolon. Various uses of the PRINT statement are shown and explained in the following examples.

ATTENTION: 150 PRINT A\$ = B\$ + C\$ WRONG
MUST BE 150 PRINT B\$ + C\$

EXAMPLES:

1. `*110 PRINT` Skips a line
2. `*110 PRINT A,B,C` Prints the value of A, B, and C separated into 16 character space zones. If the ; character were used, the A, B, and C values would be printed only one space apart.
3. `*110 PRINT "LITERAL STRING"` Prints the characters contained within the quotation marks.
4. `*110 PRINT "VALUES A & B";A,B` Prints the characters contained within the quotation marks on the line printer directly followed by variables A and B.
5. `*110 PRINT BOR(4)` Prints the number 2 on the terminal printer.

3.3 DECLARATION STATEMENTS

Declaration statements are used to specify (declare) the attributes of various variables within the BASIC program. Only one declaration statement is used in BASIC: the DIMension statement.

3.3.1 DIMension Statement

The DIMension statement allocates memory space for a memory array. One or two dimensions are allowed. The maximum array size is 255 x 255 elements. All array elements are set to zero by the DIM statement. If an array is not explicitly defined by a DIM statement, then it is implicitly assumed to be an array of 10 elements (or 10 x 10 if two elements are used) when it is first referenced in a program. The following examples show the various uses of this statement.

NOTE

The array size can only be determined once in a program, either explicitly or implicitly. In addition, only the variables A thru Z (followed by a \$ character) may be used for dimensioning string variables.

17

EXAMPLES: A(8), B(10), C(12), etc.
D(X3+8), E(Y4+10), F(Z5+12), etc.
G(10,M*2), H(9,N*3), J(10,P*4), etc.
A3(11), B4(12), C5(13), etc.
Q\$(15), R\$(20), S\$(25), etc.

Z6\$(7) - This term is not valid since only the variables A thru Z are allowed to be used with the \$ character for dimensioning string variables.

NOTE: In this example, the letter indicating the variable (i.e., X3, Y4, M*2) must be defined prior to use.

3.4 CONTROL STATEMENTS

Control Statements are used to control the natural, sequential progression of program statement execution. Control statements can be used to transfer control to another part of a program, terminate execution, or control iterative processes (loops). The following paragraphs list and explain each of the control statements.

3.4.1 FOR and NEXT Statements

The FOR and NEXT statements are used in conjunction with each other to establish program loops (iterative program processes). A loop causes the execution of one or more statements for a specified number of times.

The FOR statement consists of three parts: FOR, TO and STEP. The FOR portion specifies the initial variable at which the program begins execution. The TO portion specifies the final variable at which the program will have been completed. The STEP portion indicates the magnitude or size of each iteration (step). Subsequently, each of the statements following the FOR statement is executed. If no STEP value is specified in the FOR statement, a value of one will automatically be assumed. Although expressions are permitted for the initial, final, and incremental (step) values in the FOR statement, these three values will be evaluated only once - upon initially entering the loop.

CAUTION: FOR NEGATIVE STEPS USE STEP (-1)

The NEXT statement is located at the end of the program loop. When the NEXT statement is encountered, the variable specified by the FOR portion of the FOR statement is added to the value specified by the STEP portion. If the addition of the STEP value develops a sum that is less than the TO value (negative STEP values), the next instruction to be executed will be the first statement following the FOR statement. However, if the addition of the STEP value results in a sum that is greater than the TO value (positive STEP values) or less than the TO value (negative STEP values), then the next instruction to be executed will be the statement directly following the NEXT statement. If the

18

value of the TO portion in the FOR statement is initially less than the initial value specified in the FOR portion, the above-described loop operation will still be executed one time. When the statement following the NEXT statement is executed, the variable remaining in the FOR portion of the FOR statement will be equal to the value that caused the loop to terminate and not to the value specified by the TO portion of the statement. In the example shown below, the I variable remaining after loop termination would be equal to 9.25. It is not possible to use the same indexed variable in two different loops if the two loops are nested (embedded in a subroutine).

```
EXAMPLE:  #110 FOR I=.5 TO 10 STEP 1.25
           #120 INPUT X
           #130 PRINT I,X,X/2.75
           #140 NEXT I
```

3.4.2 STOP Statement

The STOP statement causes the program to stop executing and return the BASIC Interpreter program to the command mode. This statement differs from the END statement in that it causes the BASIC program to print the statement number where the program halted. The message printed is formatted as: "STOP XXXX". In addition, the program can be restarted by a CONTINUE command or a GOTO statement (in this case, used as a command).

3.4.3 END Statement

The END statement causes the program to stop executing and return the BASIC Interpreter program to the command mode. The END statement is similar to the STOP statement (refer to paragraph 3.4.2). In this version of BASIC, the END command may appear more than once in a program or need not appear at all.

3.4.4 GOTO Statement (Direct)

The GOTO statement causes the BASIC program to jump to the statement number specified by the GOTO statement. Program flow then continues in the normal manner from the new statement number.

```
EXAMPLE:  #220 GOTO 300
           #230 GOTO 400
           #300 E=(A+B)/C
           #310 PRINT "E=";E
```

3.4.5 GOSUB Statement

The GOSUB statement causes the program to execute a subroutine program. A subroutine program is a sequence of instructions that performs some task having utility in more than one place. Upon completion of subroutine execution, the program returns to the statement directly

following the GOSUB statement by executing a RETURN statement. The RETURN statement must be the last statement of the subroutine program. Subroutines may also be nested within other subroutines - that is, one subroutine program may use the GOSUB statement to call another subroutine program, which will then call another subroutine program. Subroutine nesting is limited to eight levels. In the following example, a short subroutine program is presented to show the use of both the GOSUB statement and the RETURN statement.

EXAMPLE:	<u>MAIN PROGRAM</u>	<u>SUBROUTINE PROGRAM</u>
	#110 X=2.5	#210 Y=(X+3)/3.1415926
	#120 GOSUB 210	#220 RETURN
	#130 PRINT Y	
	#140 STOP	

3.4.6 RETURN Statement

The RETURN is used at the end of a subroutine program to return the program to the statement directly following a GOSUB statement.

3.4.7 ON Statement

The ON statement transfers the program to another statement or subroutine as defined by the expression following the ON statement. The expression will be evaluated, truncated (cut off after the decimal point), and the program then transferred to the statement number indicated by the ON statement. Example A presents the ON statement used to jump to another statement number upon evaluation of the expression within the ON statement. Example B presents the same information as Example A with one exception: instead of jumping to another statement number, it jumps into a subroutine. Both of these examples are explained. For further information about the GOTO and GOSUB statements, refer to paragraphs 3.4.4 and 3.4.5, respectively.

EXAMPLE A: 110 ON N GOTO 200,300,550,710

EXAMPLE B: 110 ON (N+5)*3 GOSUB 200,300

Where: N = the integer value of the expression

If: N = less than 1, error occurs

If: N = more than 4 (Example A), error occurs

If: N = more than 2 (Example B), error occurs

If: N = 1, GOTO statement number 200

Then: Program continues to last N

CAUTION : ON I GOSUB is NOT correct

*This must be an expression like I * 1*

3.4.8 IF Statement (BASIC Statement - Direct)

The IF statement is used to control the execution sequence of the program statements, depending upon specific conditions. The IF statement operates in conjunction with various relational operators to determine the specific conditions that will cause the program to be advanced to the next program statement to be executed.

The relational operators listed below are used to evaluate the results of an arithmetic operation. When comparing the results of an arithmetic operation to a defined value, the arithmetic operations take precedence in their usual order (refer to paragraph 3.5.2) over the relational operators - that is, arithmetic operations are performed first and relational operators are performed last.

RELATIONAL OPERATORS

- | | | |
|----|----|-----------------------|
| 1) | = | Equal |
| 2) | <> | Not equal |
| 3) | < | Less than |
| 4) | > | Greater than |
| 5) | <= | Less than or equal |
| 6) | >= | Greater than or equal |

If the relational expression specified in the IF statement is "true", then program control is given to the statement number specified by the THEN portion of the statement. However, if the relational expression is "false", program execution continues at the next statement following the IF statement. The example provided below presents several types of IF statements.

It is also possible to include a BASIC statement after the THEN portion of the IF statement. If a BASIC statement is included and the relational expression is "true", the BASIC statement will be executed. After the BASIC statement has been executed, the user program will continue execution at the statement following the IF statement.

EXAMPLE:

```
#110 IF A<B+4 THEN 150
#120 IF A=B+4 THEN PRINT "VALUE A =" ; A
#130 IF A>B+4 THEN Q=SQR(P)+300
```

3.5 ASSIGNMENT STATEMENTS

Assignment statements are used to assign values to variables, compose and control the execution of mathematical expressions, and control the concatenation of string variables. The following paragraphs list and explain each of the assignment statements.

3.5.1 LET Statement (Direct)

The LET statement is used to assign a value to a variable. The use of the word LET is optional unless being used in the direct execution mode. The following example shows typical uses of the LET statement.

```
EXAMPLE:      #110 LET C=6553
               #120 M=(X*Y)/3*A
               #130 D=(A*B)/C
               #140 C$="YES"
               LET A=36
```

The equal sign used in the above example does not mean equivalence, as normally used in the mathematical sense. Instead, it represents a replacement operator. When used in this context, the equal sign means: replace the value of the character on the left with the value of the expression on the right. The expression on the right can be: a simple numeric value, an expression composed of numeric values, variable expressions, mathematical operators, or a string variable (as shown in line 140 of the above example).

3.5.2 Mathematical Operator Statements

The mathematical operator statements or operators used to form mathematical expressions are shown below. No two mathematical operators may occur in sequence (with one exception) and no mathematical operator may ever be assumed. Thus, A++B and (A+3) (B-4) are not valid mathematical operator statements. The only exception to this rule is in the use of exponents. In this case, A^-5 is a valid mathematical operator statement meaning that A is raised to the -5 power.

^	means	Exponentiation (raising an expression to a power)
- (unary)	means	Negate (requires only one operand)
*	means	Multiplication
/	means	Division
+	means	Addition
-	means	Subtraction

3.5.3 Operation Priority Statements

Operation priority statements control the priority of executing mathematical operator statements. This priority is listed below showing the highest priority first (parenthetical operators) followed by mathematical operators.

1. (^)
2. (-)
3. (*) or (/)
4. (+) or (-)
5. Exponentiation (raising an expression to a power)
6. Negation
7. Multiplication or division
8. Addition or subtraction

Note that in the priority structure shown above, mathematical operators contained within parentheses are executed first, followed by non-parenthetical operators. Some typical examples of both parenthetical and non-parenthetical operators are shown below.

```

:130 M=(X*Y)/3+A
:140 X=(A+B)^2

```

In the first example shown (statement number 130), the expression within parentheses will be executed first ($X \times Y$). The result of this execution will then be divided by 3. The results of these executions will then be multiplied by A. In the second example (statement number 140), the expression $A \times B$ will be executed first and this result will then be raised to the second power.

3.5.4 String Concatenation Statements

String concatenation statements are used to concatenate (join together) a series of string variables for printing on either the terminal printer or line printer. Even though a string variable may contain up to 18 characters, it may be desirable to combine several string variables for printing. When the string concatenation symbol (+ character) is used, a total of 128 characters may be combined. Only four string variables will be printed on each line, with each string variable printed in one of the four 16-character space zones. The string concatenation statement must be entered from left to right in the order you wish to have the string variables printed. Note that in Example A, the first five statements (statement numbers 110 through 150) each consists of a single string variable. Thus, if any of these string variables were executed separately (A\$ through E\$), only the word associated with that specific statement would be printed.

However, if the string concatenation statement shown in statement number 160, Example A, was executed (F\$), then the words "HI DAVE HOW ARE" would be printed on the first line while the word "YOU" would be printed on the next line (refer to Example B).

EXAMPLE A:

```

0110 A$="HI"
0120 B$="DAVE"
0130 C$="HOW"
0140 D$="ARE"
0150 E$="YOU"
0160 F$=A$+B$+C$+D$+E$

```

EXAMPLE B:

```

HI           DAVE           HOW           ARE
YOU

```

3.6 REMARKS STATEMENTS

Remarks statements are used to add comments to the users program. Only one remarks statement is used in BASIC: the REM statement.

3.6.1 REM Statement

The REM statements causes the user comment following the statement to be printed on the terminal printer. An REM statement may not contain more than 72 characters per line, including spaces (refer to paragraph 4.5.1).

CHAPTER 4

COMMANDS

4.1 INTRODUCTION

Commands (and statements used as commands) are statements which, when issued at the terminal keyboard by the user, are immediately executed by the BASIC program. All commands are issued without the need for statement numbers. The following paragraphs list each of the commands used with the BASIC program and describe the operation of each. The PRINT, LET, GOTO, and IF statements (previously discussed in Chapter 3) may also be used as commands if the system is in the command mode (waiting for an entry from the operator).

4.2 SYSTEM COMMANDS

System commands are used to immediately input or output data from or to a system peripheral device and to immediately change the contents of a specified memory location. The system commands are described in the following paragraphs.

4.2.1 LOAD Command

The LOAD command clears the memory and automatically loads into memory a user-written BASIC source program previously saved on either cassette, paper tape, or diskette.

4.2.2 APPEND Command

The APPEND command operates in exactly the same manner as the LOAD command (refer to paragraph 4.2.1), but the memory is not cleared. The APPEND command will not operate for programs loaded from diskette.

4.2.3 SAVE Command (cassette and paper tape only)

The SAVE command is used only when the user desires to load the program currently stored in memory onto either cassette or paper tape, regardless of the storage media used to store the BASIC Interpreter program. The SAVE command causes the BASIC Interpreter program to produce the necessary commands to control the read/record mechanism.

4.2.4 EXIT Command

The EXIT command is used in either of two ways, depending on the storage media used to store the BASIC Interpreter program. If the BASIC Interpreter program is stored on cassette or paper tape, the EXIT

command returns control directly to the EXbug firmware program. If the BASIC Interpreter program is stored on diskette, the EXIT command returns control to the Disk Operating System (DOS). In this case, after the word EXIT is entered by the user, followed by a Carriage Return, the BASIC Interpreter program will print the statement SAVE (Y/N), and program execution will halt to await a user response. If the user enters a Y, then the BASIC source program written by the user and currently stored in memory will be loaded onto the diskette under the user-assigned New File name (refer to Example B, paragraph 2.1). After the program is completely loaded onto the diskette, control is then returned to the DOS. If the user enters an N, the user program is cleared from memory and control is returned directly to the DOS. Entering any other character (other than a Y or N) returns control back to the BASIC Interpreter program. A user-prepared BASIC source program which is saved in this manner cannot be listed with DOS commands CLIST or PLIST, *but can be listed with LST (Do ...*

4.2.5 POKE Command

The POKE command loads a specified decimal value into a specified decimal memory address location, regardless of the data previously stored at that location. Both the decimal memory address location and the decimal value to be loaded are specified in the following manner by the user: POKE (decimal memory address location, decimal value).

CAUTION

The use of this command should be greatly limited, since a possibility of error exists. The results of an error when using this command could be quite disastrous. Not only is it possible to insert erroneous data into your memory location, but you could also unknowingly change either your program or the BASIC Interpreter program, which would render these programs useless.

4.2.6 LIST Command

The LIST command may be issued in one of three different manners: with no specification, with one specification, or with two specifications. The program statements to be listed on the terminal can be listed on a different device (such as a line printer) if the desired device is specified by the LIST command (#2 specifies the line printer). When no specific device is requested, the program statements are printed on the terminal printer.

When the LIST command is used without the optional specifications, all of the statements within the program are listed. If one of the optional specifications are included with the LIST command,

only the program statement specified by the command will be printed. However, if both of the optional specifications are used, all of the statements between the statement specified first and the statement specified last (inclusively) will be printed. The following example presents the various uses of this command along with a brief description:

EXAMPLES:

LIST	means	list all statements
LIST #2	means	list all statements on line printer
LIST 44	means	list statement number 44
LIST #2, 44	means	list statement number 44 on line printer
LIST 10, 100	means	list all statements between statement number 10 and statement number 100 (inclusively)
LIST #2, 10, 100	means	list all statements between statement number 10 and statement number 100 (inclusively) on line printer

4.2.7 TRACE ON Command

The TRACE ON command causes the terminal to print each line number enclosed within brackets as the line is executed. When used, this command provides an invaluable debugging tool, since the user can easily follow the complete program as it is being run.

4.2.8 TRACE OFF Command

The TRACE OFF command cancels the TRACE ON command (refer to paragraph 4.2.7).

4.3 PROGRAM EXECUTION COMMANDS

Program execution commands are used to execute previously-entered program statements. The BASIC program uses three program execution commands: RUN, CONTINUE, and NEW. These commands are listed and explained in the following paragraphs.

4.3.1 RUN Command

The RUN command causes the program currently stored in memory to begin execution starting at the first (or lowest) statement number. The RUN command also resets all program parameters and initializes all variables to zero.

4.3.2 CONT Command

The CONTINUE command causes the program to continue after a STOP statement (refer to paragraph 3.4.2) has been encountered in the program or a break (initiated by depressing and holding first the CTRL key and then the C key) has been entered from the terminal keyboard. This command cannot be used if an error was encountered in the program or if the program has been changed.

4.3.3 NEW Command

The NEW command causes the working storage area in memory (scratchpad memory area) and all variables and pointers to be reset. The effect of using this command is to erase all traces of the program currently stored in memory in order to start over.

4.4 EDITING COMMANDS

Even though the structure of the BASIC Interpreter program provides many ways to manipulate the user's program, four specific commands are used to edit the program while it is being entered.

4.4.1 CONTROL C Command

Depressing and holding down the CTRL key on the terminal keyboard and then depressing the C key will cause the BASIC program to halt the current operation and to respond with the word READY. After the word READY has been printed, the BASIC program will then accept further commands. The CTRL C command is often used to stop a LIST command before completion or to halt the execution of a program.

4.4.2 CONTROL X Command

Ecrire CTRL X + H= ligne à effacer

Depressing and holding down the CTRL key on the terminal keyboard and then depressing the X key will clear the current line buffer. For example, if the user types a line at the terminal keyboard and then decides that an error has been made in that line and it should be deleted, issuing a CTRL X command before the Carriage Return key is depressed will clear that line. After issuing the CTRL X command, the system responds by printing the word "DEL" on the terminal printer and moving the printer to the start of the next line.

4.4.3 RUB OUT Command

Depressing the RUB OUT key will backspace the printer one character. This permits corrections to be made as the entry is being typed. When the desired number of characters have been deleted by backspacing, the correct characters may be entered. For each backspace command, the printer will print the character _ to signify backspace. The BASIC program will prevent backspacing past the start of the line.

4.4.4 PATCH Command

The PATCH command causes the system to leave the BASIC program and return to the EXbug operating routine. When the EXbug routine is reached, the EXbug program will print the word EXBUG on the terminal printer followed by a space and the version number of the EXbug routine being used (1.1 or 1.2). The user must then enter the word MAID on the terminal keyboard. The various EXbug MAID commands may then be used (refer to the M6800 Programming Reference Manual for a complete listing of the MAID commands). When the user desires to return to the BASIC program, he enters the characters ;P, and the system will return to the step in the user program following the PATCH command. The PATCH command is often inserted as a control statement within the user's program. When encountered, control is transferred to the EXbug operating routine, permitting the user to perform changes to the program if desired. When completed, entering the ;P characters on the terminal keyboard will return the user to the next step in his program.

4.5 SIZING COMMANDS

Sizing commands are used to establish the maximum number of characters per line and the maximum number of digits to the right of the decimal point that the user will permit to be used in his program. Two sizing commands are used: LINE and DIGITS. Both of these commands are explained in the following paragraphs.

4.5.1 LINE Command

The LINE command specifies the number of character positions capable of being entered in a single line (line length) of the user's program. Each line is then automatically separated into 16 character zones by the BASIC Interpreter program. If the printer position is within the last 25 percent of the line length specified by the user and a space is entered, a Carriage Return/Line Feed operation will automatically be performed to move the printer to the start of the next line. This is done to prevent separation of a word or number between two lines. However, if this feature is not desired, it can be inhibited by setting the total line length to a number that is greater than 125 percent of the desired line length. In this manner, precise print control can be achieved. Two examples of the LINE command are presented below:

EXAMPLES:

LINE=60	means	a 60 character line length
LINE=80	means	an 80 character line length or a 60 character line length with the automatic Carriage Return/Line Feed operation inhibited.

4.5.2 DIGITS Command

The DIGITS command establishes the number of digits printed to the right of the decimal point. When this command is used, numbers greater than the number specified by the DIGITS command will automatically be truncated (cut off - not rounded) to the number of digits specified. If the number of digits entered is less than the number specified, zeros will automatically be inserted to fill the remaining digit positions. If the DIGITS command specifies a 0, the program will operate in the floating point mode. The following examples present two uses of this command.

EXAMPLES:

DIGITS=0 means reset to floating point mode.

DIGITS=6 means set to 6 decimal places. If less than six decimal places are required to print the number, the BASIC program will automatically fill the remaining positions with zeros. If more places are required, the number being printed will automatically be truncated to the number of positions specified.

CHAPTER 5

INTRINSIC FUNCTIONS

5.1 INTRODUCTION

Intrinsic functions are program functions which are inherent to the BASIC Interpreter program itself and, therefore, do not require that a separate program be prepared for execution. Four types of intrinsic functions are included in the BASIC Interpreter program: control, data, mathematical, and user defined. These functions are listed and explained in the following paragraphs.

5.2 CONTROL FUNCTIONS

The intrinsic control functions permit the user to position the print head on the terminal or line printer. Only two control functions are provided: TAB and POS. These control functions are described in the following paragraphs.

5.2.1 TAB Function

The TAB function moves the print head to the specified number of character positions to the right of the present position. This function duplicates the typewriter function of the same name. If the print head is already to the right of the position specified by the TAB command, the print head will start printing at the current print head location. The first print position (left margin) is position number 1. The last print position (right margin) is position number 72. The print head position may be expressed by any simple or string variable. More than one variable may be used if each variable is separated by a semicolon(;) character. This permits more than one print head position to be selected per line of print.

EXAMPLE:

```
110 PRINT TAB(2);B;TAB(2*R);C$
```

5.2.2 POS Function

The POS function returns the print head to the present position (in effect, the reverse of the TAB function).

EXAMPLE:

```
PRINT TAB(1);X;  
IF POS>=71 THEN PRINT
```

5.3 DATA FUNCTIONS

Various data functions are inherent to the BASIC Interpreter program that provide the user with simple access to certain mathematical operations, printer operations, and memory access operations. All of the data functions are described in the following paragraphs.

5.3.1 RND Function

The RND(X) function, when entered at the terminal keyboard by the user, produces a set of uniformly distributed pseudo-random numbers. If the RND function is used without specifying the point at which the random numbers begin (no argument used with the function), a starting point of 0 will be assumed. Then, every time the RND function is entered by the user, a different number between 0 and 1 will be printed. If a number other than zero is specified in the RND function, then a specific random number will be printed each time the RND function is entered by the user (the same number each time). If random numbers other than those between 0 and 1 are desired, then, by using the following entry, random numbers between A and B may be obtained (where A and B are the limits of the random numbers to be generated):

"PRINT INT((B-A+1)*RND(0)=A)"

5.3.2 INT Function

When the INT(X) function occurs in the program, the largest integer less than the value specified by the INT function will be printed, regardless of whether the value specified is positive or negative (refer to the example provided). The value specified by (X) may be either a constant or a variable.

EXAMPLE: M=INT(3.1415)
 N=INT(-1.414)
 Thus: M now equals 3
 N now equals -2

5.3.3 ABS Function

The ABS(X) function provides the absolute value of a number specified by the ABS function. The number specified by (X) may be either a constant or a variable.

EXAMPLE: A=ABS(3.44)
 B=ABS(-3.44)
 Thus: A now equals 3.44
 B now equals 3.44

5.3.4 SGN Function

The SGN(X) function provides the sign (+, -, or 0) of the number specified by the function. The number specified by (X) may be either a constant or a variable.

```
EXAMPLE:  X=SGN(5)      T=SGN(0)
          Y=SGN(-5)     X2=SGN(-0)
          Thus: X equals 1      T equals 0
                Y equals -1     X2 equals 0
```

5.3.5 LEN Function

The LEN(X\$) function, when entered by the user, provides the number of characters contained in the string variable specified by the function. To find the number of print positions required to print a specified number, use the following entry: LEN(STR\$(X)) where X is the specified number. The variable specified by X\$ can be either a string enclosed in quotes or a string variable.

```
EXAMPLE:  L=LEN("TESTING")
          L9=LEN("ENTER NAME")
          Thus: L now equals 7
                L9 now equals 10
```

5.3.6 ASC Function

The ASC(X\$) function provides the decimal numeric value of the first ASCII character within a string or string variable. The variable specified by (X\$) can either be a string enclosed in quotes or a string variable.

```
EXAMPLE:  D=ASC("?")
          E=ASC("A")
          Thus: D now equals 63
                E now equals 65
                B now equals 53
```

5.3.7 CHR\$ Function

The CHR\$(X\$) function provides a single character string which is equivalent to the decimal numeric value specified by the function (this is the inverse of the ASC function). The variable specified by (X\$) can either be a string enclosed in quotes or a string variable.

```
EXAMPLE:  B$=CHR$(63)
          C$=CHR$(65)
          Q$=CHR$(53)
          Thus: B$ now equals ?
                C$ now equals A
                Q$ now equals 5
```

5.3.8 VAL Function

The VAL(X\$) function provides the numeric constant equivalent to the numeric value specified by the function. The variable specified by (X\$) can either be a string enclosed in quotes or a string variable.

V=VAL(K\$) IS NOT POSSIBLE IF K\$ = (+)...

EXAMPLE: V=VAL("3.1415")
V2=VAL("-1.414")
V7=VAL("4E3")
V4=VAL("THREE")=This would produce an error message
Thus: V now equals 3.1415
V2 now equals -1.414
V7 now equals 4000

5.3.9 STR\$ Function

The STR\$(X) function provides the string value of the numeric value specified by the function (this is the inverse of the VAL function). The number specified by (X) may be either a constant or a variable.

EXAMPLE: A=12345
LET A\$=STR\$(A)
A\$ now equals the ASCII number 12345

5.3.10 LEFT\$ Function

The LEFT\$(X\$,N) function provides a string of characters from the string of characters specified by X\$, beginning with the leftmost character in the string and continuing for N characters. The string of characters specified by X\$ can be either a string enclosed in quotes or a string variable. The number of characters specified by N can be a constant or a variable.

EXAMPLE: X\$="HI DAVE HOW ARE YOU"
LET A\$=LEFT\$(X\$,6)
A\$ now equals "HI DAV"

5.3.11 RIGHT\$ Function

The RIGHT\$(X\$,N) function provides a string of characters from the string of characters specified by X\$, beginning with the rightmost character in the string and continuing for N characters. The string of characters specified by X\$ can be either a string enclosed in quotes or a string variable. The number of characters specified by N can be a constant or a variable.

EXAMPLE: X\$="HI DAVE HOW ARE YOU"
LET A\$=RIGHT\$(X\$,5)
A\$ now equals "RE YOU"

5.3.12 MID\$ Function

The MID\$(X\$,X,Y) function provides a string of characters from the string of characters specified by X\$, beginning X positions from the left and continuing for Y characters. The string of characters specified by X\$ can be either a string enclosed in quotes or a string variable. The number of characters specified by X and Y can be a constant or a variable. The use of Y is optional. If Y is not entered, the string returned is X characters from the left and continuing to the end of the string.

EXAMPLE: X\$="HI DAVE HOW ARE YOU"
LET A\$=MID\$(X\$,5,6)
A\$ now equals "AVE HO"

5.3.13 PEEK Function

The PEEK(X) function provides (in decimal) the value contained in a memory location (also in decimal) specified by the function. The number specified by X can be a constant or a variable.

EXAMPLE: LET A=PEEK(255)
A will now contain the decimal value of the data located at memory location 255₁₀.

5.4 MATHEMATICAL FUNCTIONS

Certain mathematical functions are inherent to the BASIC Interpreter program that provide the user with an easy method of performing more complex calculations than provided by other BASIC commands or statements. These calculations are listed and explained below. Note that for these functions, the answers obtained are accurate to the seventh significant digit. The accuracy of the seventh significant digit (or 1E-7 - whichever is the greater) is not guaranteed.

<u>FUNCTION</u>	<u>INTERPRETATION</u>
SIN(X)	Provides the SINE of X; where X is in radians.
COS(X)	Provides the COSINE of X; where X is in radians.
TAN(X)	Provides the TANGENT of X; where X is in radians.
ATAN(X)	Provides the Arc Tangent of X; where X is in radians.
LOG(X)	Provides the natural logarithm of X.
EXP(X)	Provides the base of the natural logarithm raised to the Xth power (this is the inverse function of LOG).
SQR(X)	Provides the square root of X.

5.5 USER DEFINED FUNCTION

In addition to the functions previously described, the BASIC Interpreter also permits the user to write and execute his own function. This function is described in the following paragraph.

5.5.1 DEF Function

The DEF function permits the user to create his own function, which can then be used within his program. To define the user's function: first, the user must enter the letters DEF; second, the user must enter the letters FN, followed directly by any alphabetic character which will identify his function; third, the user must enter any non-subscripted numeric value and this value must be enclosed within parentheses (the variable is basically a dummy variable). This user function must then be made equal to any valid expression of the user's own choosing. During execution, the dummy variable is replaced with the user expression. The same function may not be defined more than once in any program. A function must be defined before it is used. A brief program is shown in the example below to illustrate the use of the DEF function.

EXAMPLE:

```
#110 DEF FNA(X)=(3.14*X)^X
#120 DATA 5,6,7,0
#130 READ X
#140 IF X=0 THEN END
#150 PRINT FNA(X)
#160 GOTO 130
#RUN
```

```
78.5
113.04
153.36
READY
```


APPENDIX E

ERROR MESSAGES

Various errors made during program entry and execution cause the system to print an error message, which can be used to correct the error. The example presented below shows the format used by the system to notify the user of an error. Each error message is explained below.

EXAMPLE: Error #0003 in line #110

The user first locates the specified line and then finds the error specified by the message in the below list. In this example: Illegal character or variable located on line number 110.

<u>MESSAGE #</u>	<u>MESSAGE</u>
0001	Oversized variable (over 255) used with the TAB or CHR\$ function (refer to paragraphs 5.2.1 and 5.3.7, respectively) or with the ON statement (refer to paragraph 3.4.7).
0002	Input error (refer to paragraph 3.2).
0003	Illegal character or variable (refer to paragraph 1.4).
0004	No ending quotation marks (") used after characters to be printed (refer to paragraph 3.2.4).
0005	Dimensioning error (refer to paragraph 3.3.1).
0006	Illegal arithmetic (refer to paragraph 3.5.2).
0007	Line number not found.
0008	Divide by zero attempted.
0009	Excessive subroutine nesting (refer to paragraph 3.4.5).
0010	RETURN used without prior GOSUB statement (refer to paragraph 3.4.6).
0011	Illegal variable (refer to paragraphs 1.4.2 and 1.4.3).
0012	Unrecognizable statement (refer to paragraph 1.3.1).
0013	Parentheses error.
0014	Memory full.

MESSAGE #

MESSAGE

- 0015 Subscript error.
- 0016 Excessive FOR loops (maximum 8 - refer to paragraph 3.4.1).
- 0017 NEXT "X" without FOR defining "X" (refer to paragraph 3.4.1).
- 0018 FOR/NEXT statements nesting error (refer to paragraph 3.4.1).
- 0019 READ statement error (refer to paragraph 3.2.2).
- 0020 Error in ON statement (refer to paragraph 3.4.7).
- 0021 Input overflow - input line exceeds 72 characters (refer to paragraph 1.3.1).
- 0022 Syntax error in DEF function (refer to paragraph 5.5.2).
- 0023 Syntax error in function or function not defined (refer to paragraph 5.5.2).
- 0024 Error in STR\$ usage or mixing of numeric and string variables (refer to paragraph 5.3.9).
- 0025 String buffer overflow or string extract (in LEFT\$, RIGHT\$, or MID\$) too long (refer to paragraphs 5.3.9 through 5.3.12).
- 0026 I/O operation requests port without I/O module installed.

27. illegal CHR for VAL function (Ex - 14J25)
 must be .

APPENDIX F

ENHANCING PROGRAM OPERATION

When preparing a program using the BASIC Interpreter, program execution time can be reduced by observing the few simple rules listed below.

1. The use of spaces should be limited since, even though they are not processed by the system, they do consume both memory allocation and input/output time.
2. Since the BASIC Interpreter must search for subroutines and functions in the source file, the user should arrange the source file in such a manner that the most used subroutine or function is located at the start of the program, followed by the second most used subroutine or function, etc. If this is done, the time required for the BASIC Interpreter to search the source file will be reduced.
3. Since subscripted variables take considerable processing time, whenever possible, use non-subscripted variables.
4. Using the mathematical functions listed in paragraph 5.4 (SIN, COS, TAN, ATAN, EXP, and LOG) is very slow. Therefore, use these functions only when necessary. The $\wedge$ (exponentiation) operator also uses both the LOG and EXP functions, making the exponentiation operator quite slow. If you want to square a number, use A^*A .
5. The BASIC Interpreter program searches the symbol table for a referenced variable and variables are inserted into the symbol table as they are referenced. Therefore, time can be saved if a frequently used variable is called early in the program so that this variable is located near the start of the symbol table.
6. Numeric constants are converted each time they are encountered in the program. Therefore, if you use a constant often, assign it to a variable and use the variable instead.

APPENDIX G

MINIMIZING DEDICATED MEMORY

When preparing a program using the BASIC Interpreter, the size of dedicated memory (core) required for the program can be reduced by observing the few simple rules listed below.

1. Remark (REM) statements take space in memory, so use them sparingly.
2.
 - A) Each non-subscripted simple variable requires 8 bytes.
 - B) Each non-subscripted string variable requires 20 bytes.
 - C) Each numeric array requires 6 bytes plus an additional 6 bytes for each element.
 - D) Each string array requires 6 bytes plus an additional 18 bytes for each element.
3. An implicitly dimensioned variable creates a 10 element array (or 10x10). Therefore, if you do not intend to use 10 elements, save core memory by explicitly dimensioning the variable.
4. Each line entered in the program requires 2 bytes for the line number, 1 byte for the encoded key word (statement, command, or intrinsic function), 1 byte for every character following the key word, and 1 byte for the end of line terminator. By using as few spaces as possible when entering the line, core memory can be reduced.
5. The BASIC Interpreter program reserves the first 256 bytes of memory for stack and buffer use.

